

망하기 전에 터뜨려보기

웹 프로젝트 부하 테스트 실전 특강
- 관찰가능성을 코드로 만들기

Step 1: k6 10줄로 Smoke Test

왜 Smoke Test? 배포 후 1분 안에 서비스 죽었는지 확인하는 최소한의 생존 테스트.
실패 시 즉시 배포 중단.

```
import http from 'k6/http';
import { check } from 'k6';
export const options = {
  vus: 1, duration: '30s',
  thresholds: {
    'http_req_failed': ['rate<0.01'],
    'http_req_duration': ['p(95)<500']
  }
};
export default () => {
  const res = http.get('https://api.myservice.com/health');
  check(res, { 'status 200': (r)=>r.status===200 });
};
```

1 VU

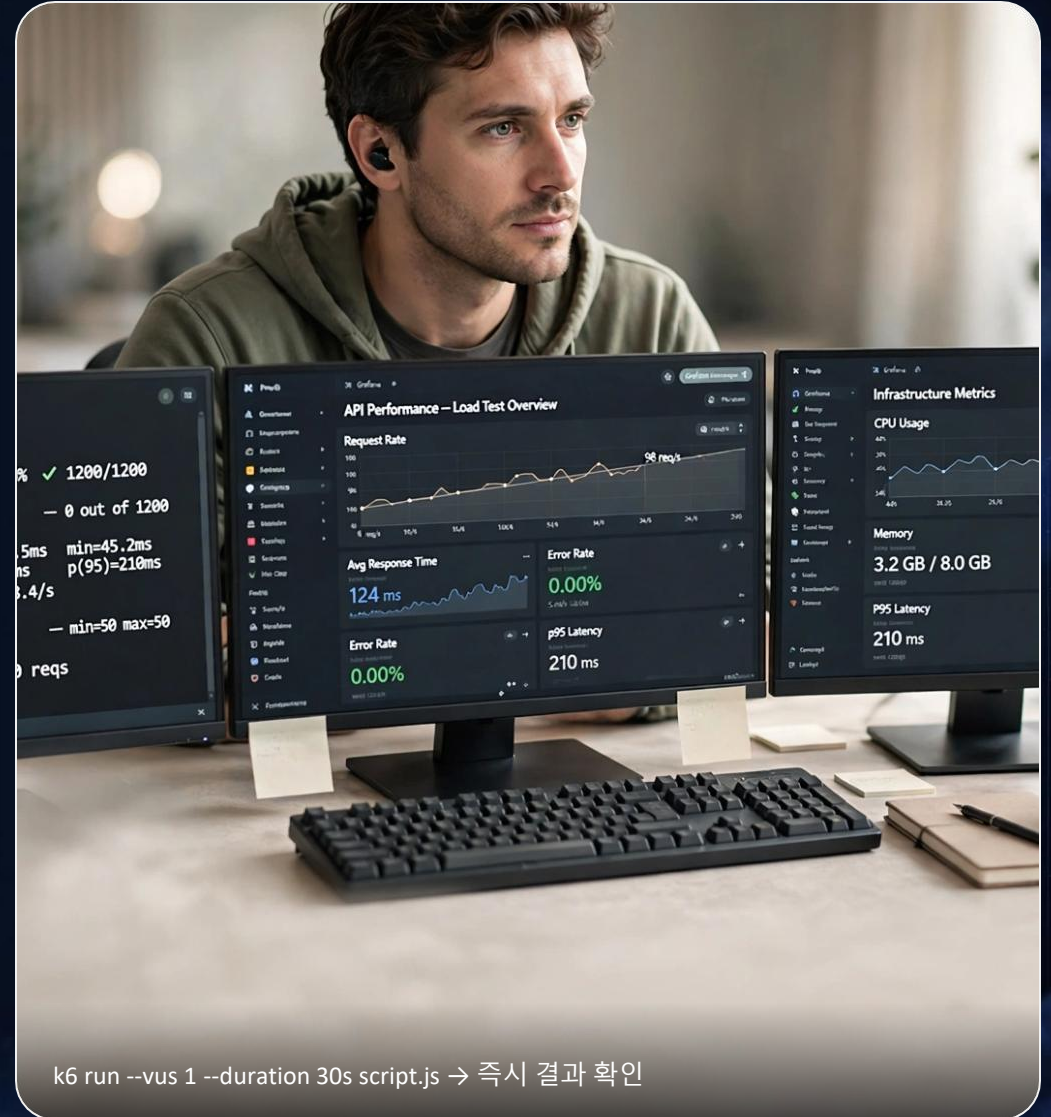
30초 최소 부하

Ê Ç Ã

http_req_failed 목표

p95

실제 사용자 체감



Step2: Prometheus + Grafana 켜기

App
/metrics :9090



Prometheus
scrape 15s



Grafana
dashboard:1860

PROMETHEUS 설정

```
scrape_configs:  
  - job_name: 'api'  
    scrape_interval: 15s  
    static_configs:  
      - targets: ['api:3000']
```

핵심 메트릭 4종

- CPU / 메모리 usage
- http_requests_total
- http_request_duration histogram
- process_resident_memory

실습 명령어

```
docker-compose up -d prometheus grafana  
  
# Grafana 접속  
http://localhost:3000  
# DataSource → Prometheus  
# Import dashboard 1860
```

체크 포인트

- ✓ /metrics 노출 확인
- ✓ Prometheus Targets UP
- ✓ Grafana 그래프 실시간 갱신



💡 실습 팁: k6 실행 중에 Grafana에서 p95 Latency가 210ms로 찍히는 걸 눈으로 확인하면 관측 시작 성공입니다.

Step3: JSON + traceId 심기

원칙 라벨은 적게, JSON 필드는 풍부하게 — Loki 카디널리티 폭주를 막기 위해 label은 service, level 만. 상세 정보는 JSON 필드로.

필수 10필드 · 로그 관측성의 최소 스펙

timestamp level service **traceId** **spanId** method path status latency
message

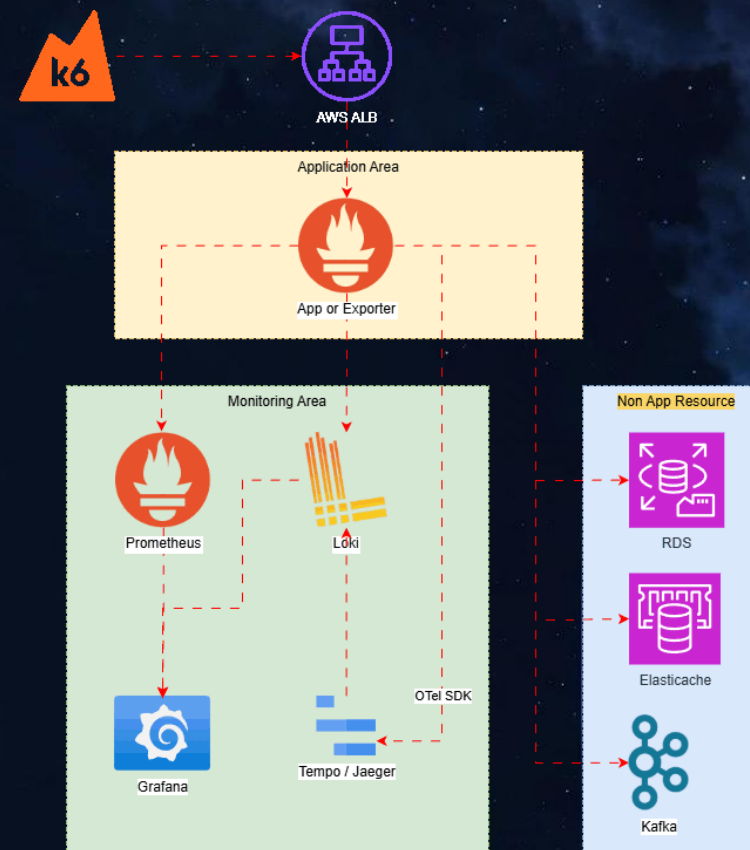
커스텀 오류 + HTTP 동시 필터

errorCode E001~ 인증실패·결제오류
status 401/500 HTTP 상태와 조합
예: 500 AND E004 → DB연결 실패 즉시 특정

실습 코드 한 줄

```
logger.info({  
  traceId, spanId,  
  errorCode: 'E001',  
  status: 401  
})
```

stdout JSON → Promtail/Vector → Loki / OpenSearch / Elasticsearch → Grafana // Datadog Agent 하나면 통합 수집



⚠ **Loki 카디널리티 주의:** traceId, userId 같은 고유값은 라벨 금지. JSON 필드에 넣어야 스토리지·성능 안전.

Step4: 로그 대시보드 1~6 — 실시간으로 장애 찾기

2026-08-07 실시간 모니터링 • JSON+traceld 심은 후 바로 보이는 6가지 뷰

1 Latency Histogram by Path
경로별 p95 분포 · /api/주문 480ms 주의

2 커스텀 오류 Top N — errorCode + HTTP status
E001 인증실패 120 & 500 서버오류 112건 조합 필터

3 Slow Query >1000ms
Q-20934 1420ms 즉시 식별 · 인덱스 누락 탐지

4 User Journey — traceld
trace 9f7b2a로 로그인→결제 5단계 1715ms 추적

5 Abnormal IP >1000
45.67.89.12 봇 1843회 · 자동 차단 룰

6 Business KPI — 결제실패율 2.4%
목표 3.0% 이내 · 실패 318건 실시간

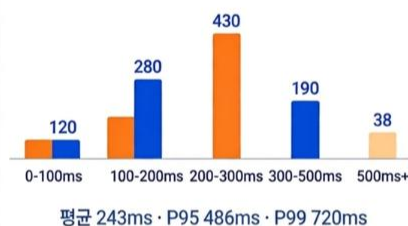


2026-08-07 실시간 모니터링

• 실시간 · 업데이트 완료



1. 응답 지연시간 히스토그램



2. 커스텀 오류 Top N - errorCode별 + HTTP status

errorCode별	HTTP status
E001 인증실패 120	500 서버오류 112
E002 결제오류 87	401 인증필요 95
E003 타임아웃 54	403 권한없음 68
E004 DB연결실패 32	404 Not Found 44
E005 외부API오류 19	



3. 느린 쿼리 (Slow Query) Top 3

SELECT orders JOIN payments ... 실행시간 2.3s · 1,243회 호출	⚠ 위험
SELECT user JOIN login_log ... 실행시간 1.8s · 892회 호출	지연
SELECT product WHERE category = ? ... 실행시간 1.4s · 654회 호출	관찰



4. 사용자 여정 traceld



5. 비정상 IP 탐지

⚠ 45.67.89.12 봇 탐지 · 214회 차단 · 러시아
⚠ 103.22.45.77 무차별대입 · 132회 차단 · 베트남
i 198.51.100.23 비정상 접근 · 87회 차단 · 미국

금일 차단 IP 총 433건 · 차단규칙 12개 활성화



6. 비즈니스 KPI

DAU 12,458 +5.2% ↑	매출 8.2백만원 +3.1% ↑
주문 건수 1,024건 +4.8% ↑	% 전환율 3.6% +0.4%p ↑

데이터 기준: 2026-08-07 14:35 KST · 자동 갱신 30초

필터 예시: {status:500 AND errorCode:E004} → DB 커넥션 풀 원인 즉시 특정

Step5: 로그 대시보드 7~12 — 고급 관측

2026-08-07 실시간 모니터링 · 장애 전조부터 비용까지 한눈에

07

Cache Miss Storm

캐시 미스 폭풍 탐지

1,240건 ↑ 42% · 피크 32%

08

External Failure Map

외부 API 장애 지도

APAC 3 · EU 1 · US 2

09

GC / OOM 전조

힙 메모리 / GC 경보

힙 87% · 경보 높음

10

Security 401/403 Top IP

공격 IP 차단

192.168.1.102 123건 차단

11

Deployment Correlation

배포 버전별 오류율

v2.4.1 → 오류 12%

12

Log Volume Cost

로그 볼륨 · 비용 모니터링

1.2TB · 34만원 (+12%)



2026-08-07 실시간 모니터링

실시간 · 정상 감시 중

로그 모니터링 대시보드 · 6가지 지표를 한눈에 확인

07

Cache Miss Storm

캐시 미스 폭풍



발생 건수 1,240건 ↑ 42%

피크 시간 14:22 · 최근 1시간 동안 +520건

08

External Failure Map

외부 장애 지도



● APAC · 3건
● EU · 1건
● US · 2건

영향 서비스: 결제API, 인증API · 평균 복구 6분

09

GC/OOM 전조 알림

GC/OOM 전조 알림



메모리 사용률 87%

경고 3건 · 힙 메모리 안정화 필요

경보 수준: 높음

10

Security 401/403 Top IP

보안 이상 IP



192.168.1.102 · 123건 401: 98건
203.0.113.45 · 98건 403: 61건
198.51.100.22 · 76건 401: 55건

차단 규칙 적용: 자동

11

Deployment Correlation

by version

배포 상관관계

v2.4.1 · 오류율 12% v2.4.0 · 오류율 3% v2.3.9 · 오류율 1%

배포 시각 13:00 · 상관관계 높음

12

Log Volume Cost

로그 볼륨 비용



1.2TB · 340,000원

전일 대비 +12% ↑ 36,000원

스토리지 58% 전송 42%

✓ 정상

업데이트: 2026-08-07 14:23 · 자동 수집 간격 1분

데이터 기준: 최근 1시간

관측의 완성 - Grafana Exemplars

Exemplars란?

메트릭 그래프의 점 하나에 **TraceID**를 박아두는 것. 이상치 클릭 한 번으로 원인까지 점프.

MetricPrometheus
Grafana 이상치 클릭



Tracespan → TraceID 확인



Logtraceld로 로그 조회

설정: Prometheus enableExemplars + Grafana Exemplars 활성화 + OpenTelemetry SDK

효과: p95가 튀는 순간, 해당 요청 로그까지 **3초 컷 점프** — MTTR 70% 단축

```
exemplars:  
  enabled: true # prometheus.yml  
  # Grafana Datasource → Exemplars ON  
  traceld injection via OTEL SDK
```

Step6: p95 < 500ms CI 게이트 - 성능 기준 자동화

왜 평균이 아니라 p95?

사용자 체감 기준

평균 120ms면 괜찮아 보여도, 상위 5%는 1.8초 지옥을 경험합니다. p95는 “95%의 사용자가 이 시간 안에 응답받음”을 보장하는 실전 지표입니다.

120ms

평균 - 낙관적

486ms

p95 - 실제 체감

임계값 철학: p95 < 500ms, 실패율 < 1% - 두 조건 모두 만족해야 통과. 하나라도 실패 시 배포 중단.

CI에서 자동 차단하기

```
export const options = {
  thresholds: {
    'http_req_duration': ['p(95)<500'],
    'http_req_failed': ['rate<0.01']
  }
};
```

PIPELINE

build



k6 smoke



p95 체크



Grafana annotate



deploy

GitHub Actions: k6 run 시 exit code 99면 실패. if threshold fails, block deploy
실습: thresholds 100ms로 일부러 낮춰서 실패시켜 보기 - 빨간 로그가 CI를 막는 경험.

Step7: 체크리스트 10선 - 부하테스트 전 반드시 확인

하나라도 빠지면 결과가 무효 - 시작 전 2분 체크리스트

- | | |
|---------------------------------------|---|
| 1 동일스펙?
환경 / 리소스 / 설정 일치 여부 | 2 웜업?
사전 웜업 수행 여부 |
| 3 Mock?
외부 의존성 Mock/Stub 여부 | 4 ThinkTime?
사용자 지연 / 대기시간 적용 |
| 5 중단기준?
통과·실패 기준 정의 여부 | 6 지표수집?
CPU / 메모리 / 네트워크 수집 |
| 7 샘플링 설정?
Trace 샘플링 비율 확인 | 8 리소스 모니터링?
대상 시스템 감시 여부 |
| 9 에러율 확인?
오류율 / 지연 임계값 검토 | 10 결과 비교/검증?
베이스라인과 비교 여부 |

✅ 팁: 이 10개는 CI 파이프라인 시작 조건으로 넣으세요 - 체크 안 되면 k6 실행 자체를 막기



Metric

Prometheus/Grafana
Examples 활성화



Trace

분석추적 연결
Span → TraceID 확인



Log

로그로 점프
Trace에서 Log 조회

10/10

체크 완료 시 유효한 테스트

0

누락 허용 항목

Step8: 실수 Top5 — 절대 하지 말아야 할 것

2교시 실습에서 가장 많이 터지는 사고 사례 • 하나만 피해도 3시간 세이브

1 운영DB 직격

프로덕션 DB에 그대로 부하 테스트 실행 → 실제 고객 결제 지연 · 락 폭주 장애

→ 격리된 스테이징 DB + 동일 스펙 복제

2 로컬 1만 VU

내 노트북에서 10k VU → 부하기 CPU 100%, 네트워크 포화 → 서버는 정상인데 테스트만 실패

→ 분산 부하 · k6 cloud / 클러스터

3 평균만 보기

avg 120ms만 보고 통과 → p95 720ms, 상위 5% 사용자는 이탈

→ p95/p99 · 히스토그램 필수 확인

4 부하기 미모니터링

k6 자체 CPU · 메모리 · http_req_failed 미확인 → 병목이 API인지 부하기인지 구분 불가

→ k6 메트릭 + Grafana 부하기 대시보드

5 비용 폭탄

Loki 고카디널리티 라벨 + 전체 로그 수집 → 1.2TB/일 → 34만원 · 전일대비 +12%

→ 샘플링 · 라벨 최소화 · 비용 알람

예방 5계명 격리 환경 · 분산 부하 · 분포값 확인(p95) · 부하기 모니터링 · 비용 알람 설정 — 체크리스트 10선과 함께 실습 전 반드시 확인

Q&A + 자료공유 — 마무리

자주 묻는 질문

Q. 라벨에 traceId 넣으면 안 되나요?

A. Loki 카디널리티 폭주 → JSON 필드로. 라벨은 service, level 2개로 최소화.

Q. p95가 평균보다 왜 중요한가요?

A. 5% 사용자는 평균 뒤에 숨은 1초+ 지연을 겪습니다. 체감은 p95.

Q. Datadog vs OSS 선택 기준은?

A. 초기 빠른 통합은 Datadog, 비용 효율 + 커스텀은 Loki+Grafana.



k6 10줄 템플릿

smoke.js thresholds 포함

```
k6 run --vus 1 --duration 30s script.js
```



Prometheus + Grafana

docker-compose + datasource.yml

```
compose up -d prometheus grafana / import 1860
```



로그 JSON 포맷

10필드 + 커스텀 errorCode

```
{timestamp, level, traceId, errorCode, status, latency, message}
```



대시보드 JSON 12종

1~6 커스텀 오류, 7~12 고급 관측

```
log_dashboard_custom_2026.json / 7_12_2026.json
```

NEXT ASSIGNMENT

내 서비스에 traceId 심고 p95 < 500ms CI 게이트 PR 올리기 →
Grafana 어노테이션으로 증명

2교시 마무리 · CLOSING

오늘 실습으로 얻는 것

Observability 확보

 k6 10줄로 내 서비스 Smoke Test 가능 — 배포 후 1분 생존 확인

 Prometheus + Grafana로 메트릭 보기 — p95 486ms 실시간 추적

 JSON + traceId + errorCode로 로그 점프 — Metric → Trace → Log 3초 완성

 p95 < 500ms로 CI에서 자동 차단 — 성능 기준 자동화

관찰가능성 확보 + 성능 기준 CI 자동화 완성